



Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования «Московский государственный технический
университет имени Н.Э. Баумана национальный исследовательский
университет» (МГТУ им. Н.Э. Баумана)

Модульное домашнее задание №1

Вариант 12

Методы кодирования цифровых данных

Студент группы ИУ1-31Б

Соин А. Д.
«17» апреля 2026 г.

Преподаватель

Кузнецов М. А.
«_____» _____ 2025 г.

Москва, 2026

Содержание

1	Методы циклического кодирования	2
1.1	Кодирование исходной последовательности	2
1.2	Построение таблицы синдромов	2
1.3	Моделирование ошибки	2
2	Применение методов физического кодирования	2

1 Методы циклического кодирования

1.1 Кодирование исходной последовательности

Порождающий полином: $g(x) = x^5 + x^2 + 1$, в двоичном виде $g(x) = 100101$.

Степень полинома: $r = n - k = 5$.

Сообщение: $46_{10} = 101110_2$.

Добавляем избыточность: $m(x) = 10111000000_2$

Деление сообщения на полином: $R = 00010_2$.

Итоговое кодовое слово $c = 10111000010_2$.

1.2 Построение таблицы синдромов

i	x^i	R
0	00000000001	00001
1	00000000010	00010
2	00000000100	00100
3	00000001000	01000
4	00000010000	10000
5	00000100000	00101
6	00001000000	01010
7	00010000000	10100
8	00100000000	01101
9	01000000000	11010
10	10000000000	10001

1.3 Моделирование ошибки

Исходное сообщение: $c = 10111000010_2$.

Сообщение с ошибкой: $c = 10111010010_2$.

Синдром для данного сообщения: $S(x) = c \bmod g(x) = 10000_2$.

По таблице синдромов ошибка $i = 4$.

Для исправления ошибки произведем операцию $10111010010_2 \oplus 00000010000_2 = 10111000010_2$.

Это значение соответствует исходному сообщению, следовательно кодирование проведено верно.

2 Применение методов физического кодирования

Для построения временных диаграмм был реализован следующий код matlab.

```
clc; clear; close all;

%% ----- Путь сохранения -----
scriptPath = fileparts(mfilename('fullpath'));
if isempty(scriptPath)
    scriptPath = pwd;
end

outDir = fullfile(scriptPath, 'results');
if ~exist(outDir, 'dir')
    mkdir(outDir);
end

%% ----- Исходные данные -----
data = [0 0 1 0 1 1 1 0]; % 46 = 00101110 (MSB → LSB)
```

```

Tb = 1;
samples = 100;

t = 0:Tb/samples:Tb*length(data)-Tb/samples;

expand = @(bits) repelem(bits, samples);

%% ----- NRZ (однополярный) -----
nrz_uni = expand(data);

figure; stairs(t, nrz_uni, 'LineWidth', 1.5);
title('NRZ Unipolar'); grid on;
ylim([-0.5 1.5])
exportgraphics(gcf, fullfile(outDir, 'NRZ_unipolar.png'), 'Resolution', 300);

%% ----- NRZ (биполярный) -----
nrz_bi = expand(2*data - 1);

figure; stairs(t, nrz_bi, 'LineWidth', 1.5);
title('NRZ Bipolar'); grid on;
ylim([-1.5 1.5])
exportgraphics(gcf, fullfile(outDir, 'NRZ_bipolar.png'), 'Resolution', 300);

%% ----- NRZ-I (инверсия при 1) -----
level = -1;
nrz_i1 = zeros(1, length(data));
for i = 1:length(data)
    if data(i) == 1
        level = -level;
    end
    nrz_i1(i) = level;
end
nrz_i1 = expand(nrz_i1);

figure; stairs(t, nrz_i1, 'LineWidth', 1.5);
title('NRZ-I (инверсия при 1)'); grid on;
ylim([-1.5 1.5])
exportgraphics(gcf, fullfile(outDir, 'NRZ_I_1.png'), 'Resolution', 300);

%% ----- NRZ-I (инверсия при 0) -----
level = -1;
nrz_i0 = zeros(1, length(data));
for i = 1:length(data)
    if data(i) == 0
        level = -level;
    end
    nrz_i0(i) = level;
end

```

```

nrz_i0 = expand(nrz_i0);

figure; stairs(t, nrz_i0, 'LineWidth', 1.5);
title('NRZ-I (инверсия при 0)'); grid on;
ylim([-1.5 1.5])
exportgraphics(gcf, fullfile(outDir, 'NRZ_I_0.png'), 'Resolution', 300);

%% ----- AMI -----
ami = zeros(1, length(data));
level = 1;
for i = 1:length(data)
    if data(i) == 1
        ami(i) = level;
        level = -level;
    else
        ami(i) = 0;
    end
end
ami = expand(ami);

figure; stairs(t, ami, 'LineWidth', 1.5);
title('AMI'); grid on;
ylim([-1.5 1.5])
exportgraphics(gcf, fullfile(outDir, 'AMI.png'), 'Resolution', 300);

%% ----- RZ -----
rz = [];
for i = 1:length(data)
    if data(i) == 1
        rz = [rz ones(1, samples/2), zeros(1, samples/2)];
    else
        rz = [rz -ones(1, samples/2), zeros(1, samples/2)];
    end
end

figure; stairs(t, rz, 'LineWidth', 1.5);
title('RZ'); grid on;
ylim([-1.5 1.5])
exportgraphics(gcf, fullfile(outDir, 'RZ.png'), 'Resolution', 300);

%% ----- Manchester -----
man = [];
for i = 1:length(data)
    if data(i) == 0
        man = [man ones(1, samples/2), -ones(1, samples/2)];
    else
        man = [man -ones(1, samples/2), ones(1, samples/2)];
    end
end
end

```

```

figure; stairs(t, man, 'LineWidth', 1.5);
title('Manchester (IEEE 802.3)'); grid on;
ylim([-1.5 1.5])
exportgraphics(gcf, fullfile(outDir, 'Manchester.png'), 'Resolution', 300);

%% ----- Differential Manchester -----
diff_man = [];
level = -1;
for i = 1:length(data)
    if data(i) == 0
        level = -level;
    end
    first = level;
    second = -level;
    diff_man = [diff_man ...
        first*ones(1, samples/2), ...
        second*ones(1, samples/2)];
    level = second;
end

figure; stairs(t, diff_man, 'LineWidth', 1.5);
title('Differential Manchester'); grid on;
ylim([-1.5 1.5])
exportgraphics(gcf, fullfile(outDir, 'Diff_Manchester.png'), 'Resolution', 300);

%% ----- 2B1Q -----
pairs = reshape(data, 2, [])';
map = containers.Map({'00', '01', '10', '11'}, [-2.5 -0.833 0.833 2.5]);

levels = zeros(1, size(pairs,1));
for i = 1:size(pairs,1)
    key = num2str(pairs(i,:), '%d%d');
    levels(i) = map(key);
end

b2q = repelem(levels, samples);
t2 = 0:Tb/samples:Tb*length(levels)-Tb/samples;

figure;
stairs(t2, b2q, 'LineWidth', 1.5);
title('2B1Q'); grid on;
ylim([-3.5 3.5])
exportgraphics(gcf, fullfile(outDir, '2B1Q.png'), 'Resolution', 300);

```

В результате были получены следующие графики

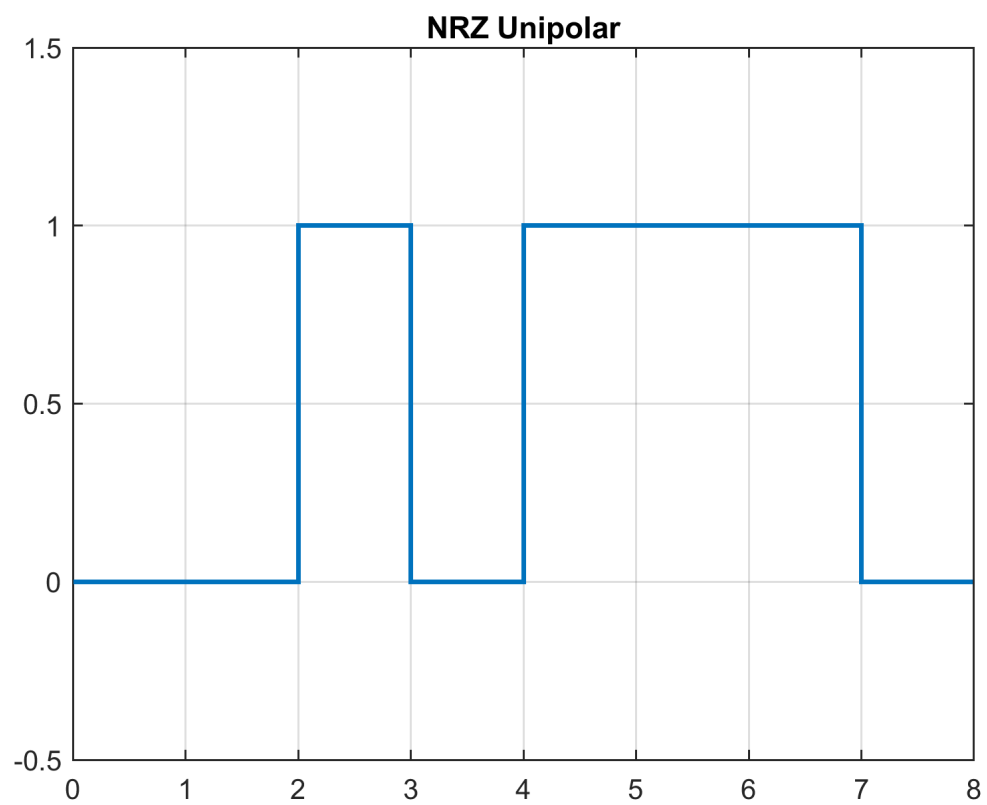


Рис. 1. Код без возвращения к нулю (однополярный)

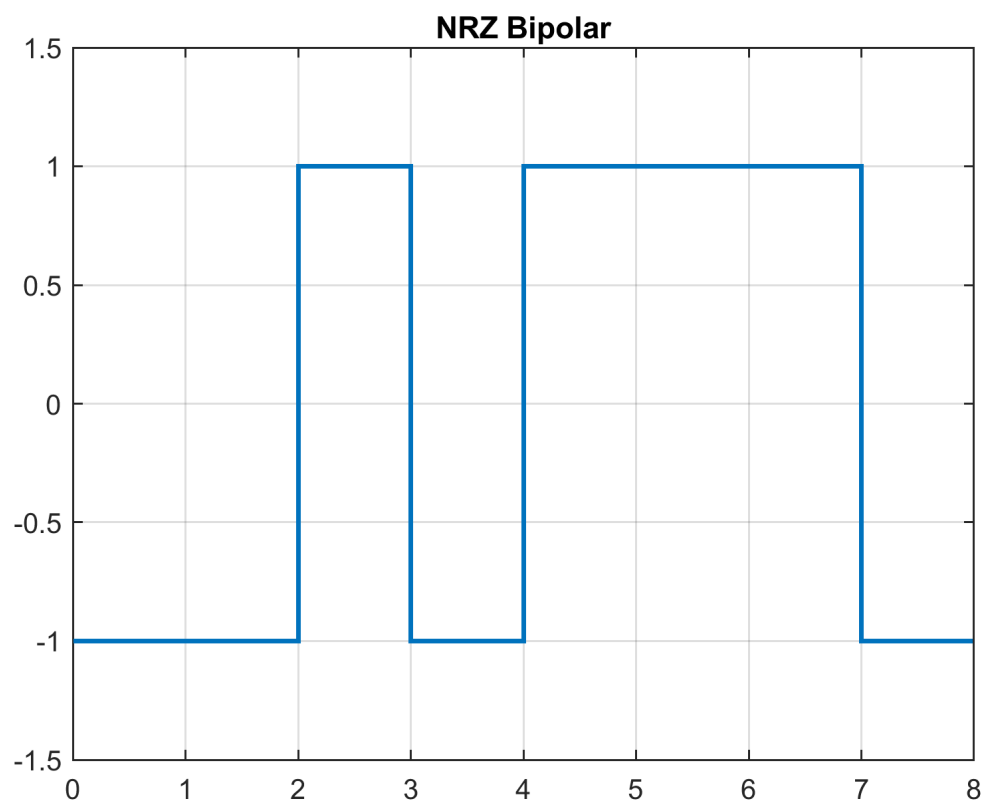


Рис. 2. Код без возвращения к нулю (биполярный)

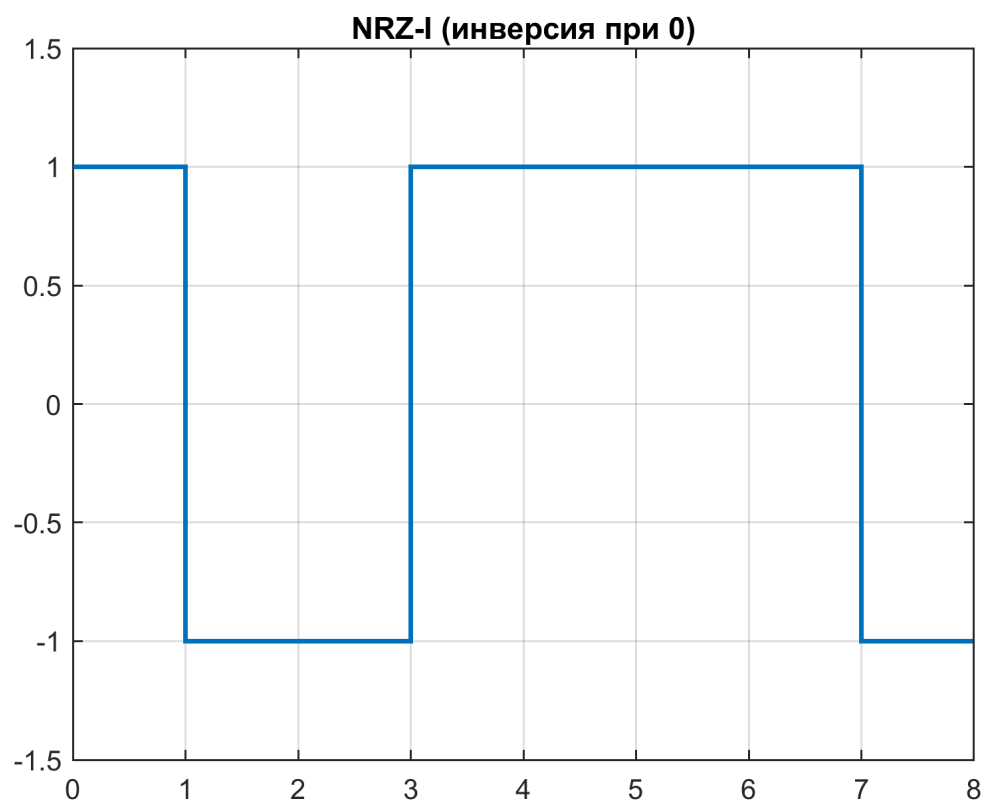


Рис. 3. Код без возвращения к нулю с инверсией при нуле

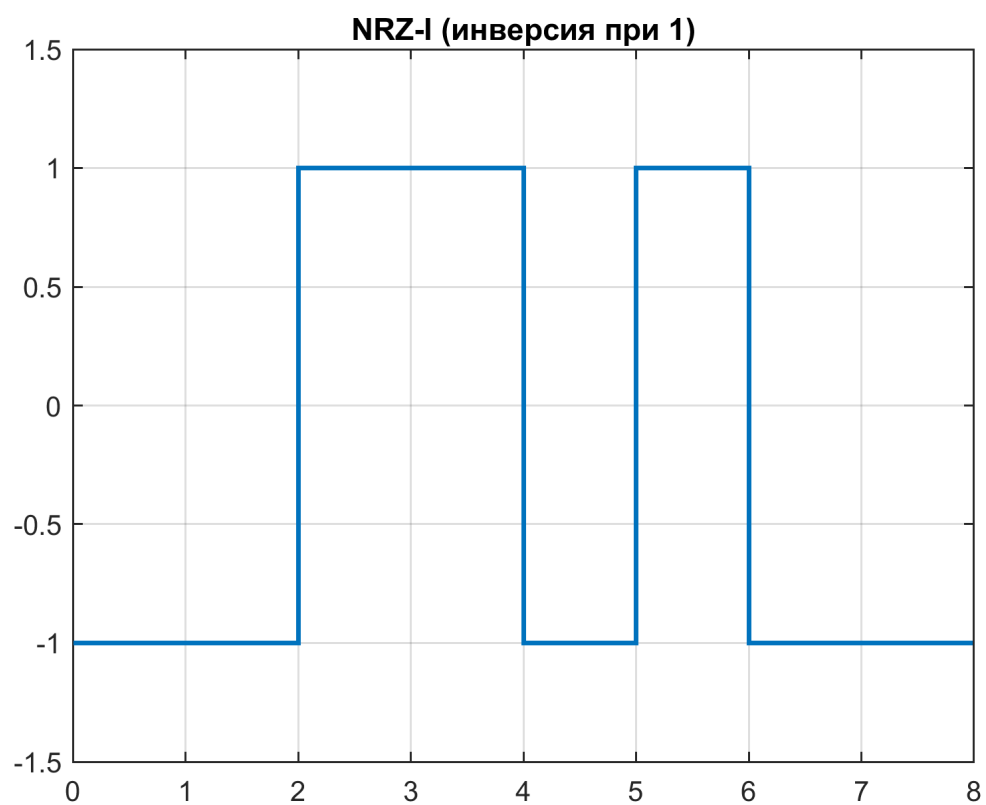


Рис. 4. Код без возвращения к нулю с инверсией при единице

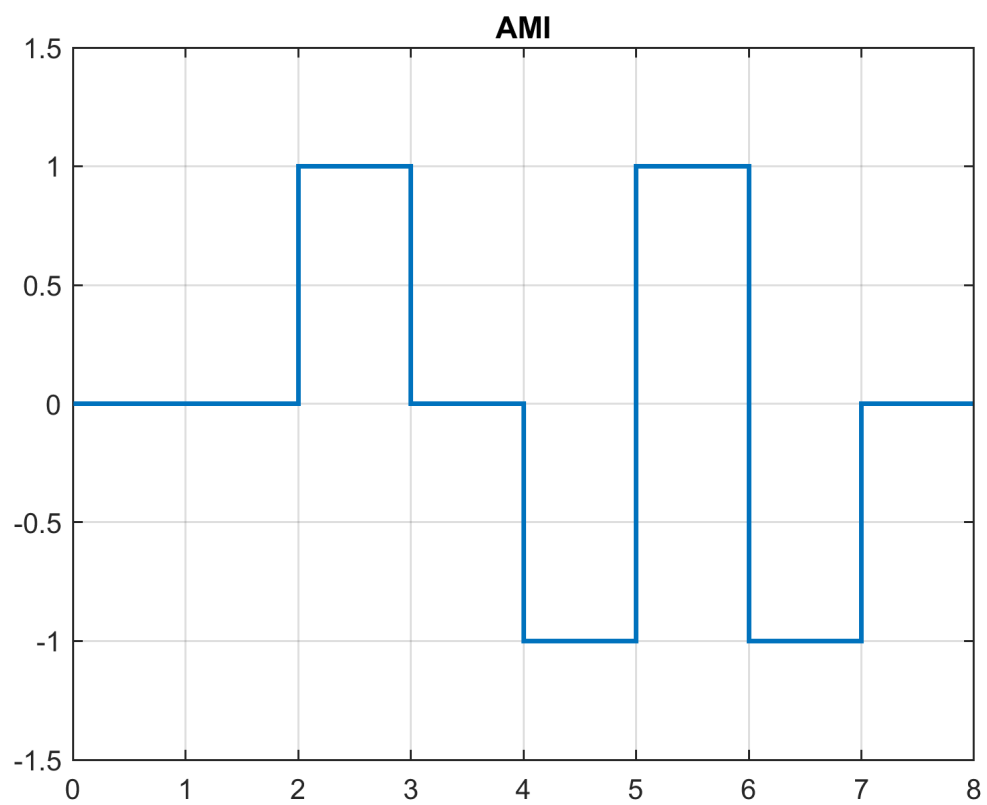


Рис. 5. Код с альтернативной инверсией

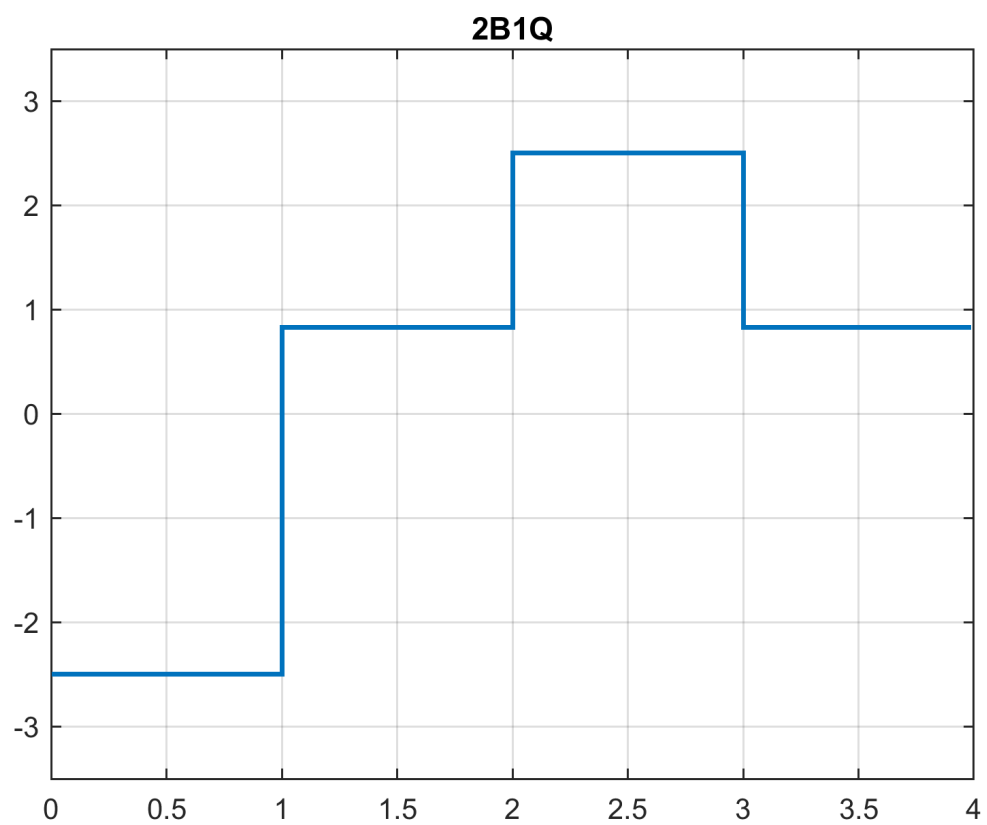


Рис. 6. Код 2B1Q

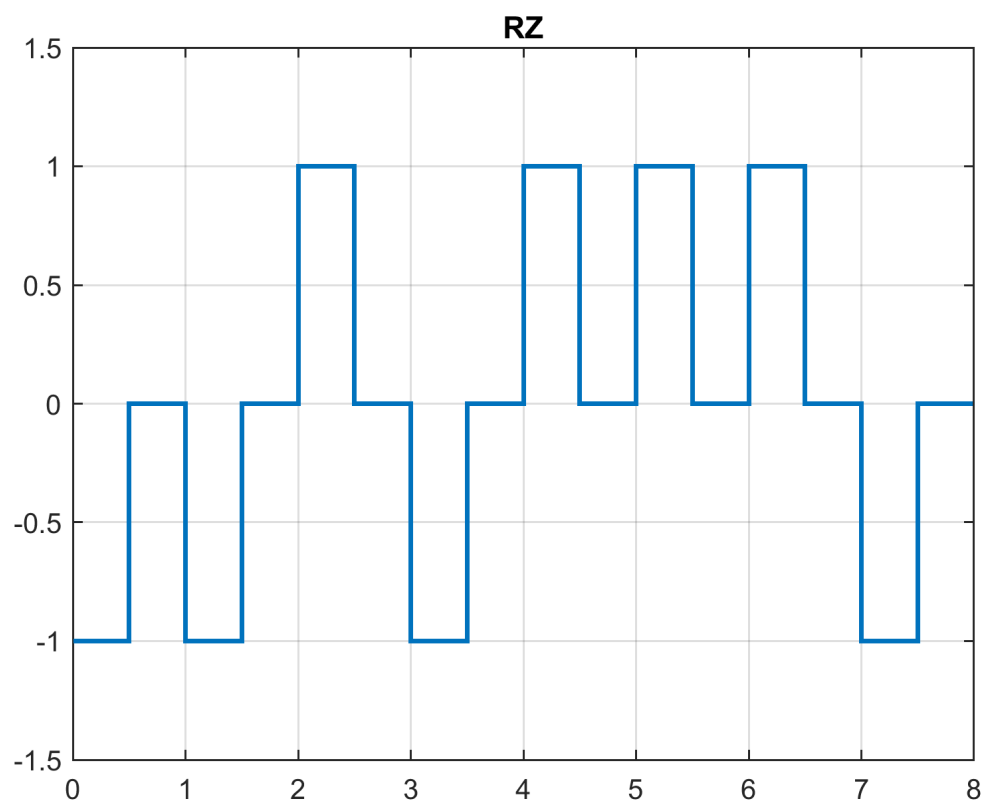


Рис. 7. Код с возвращением к нулю

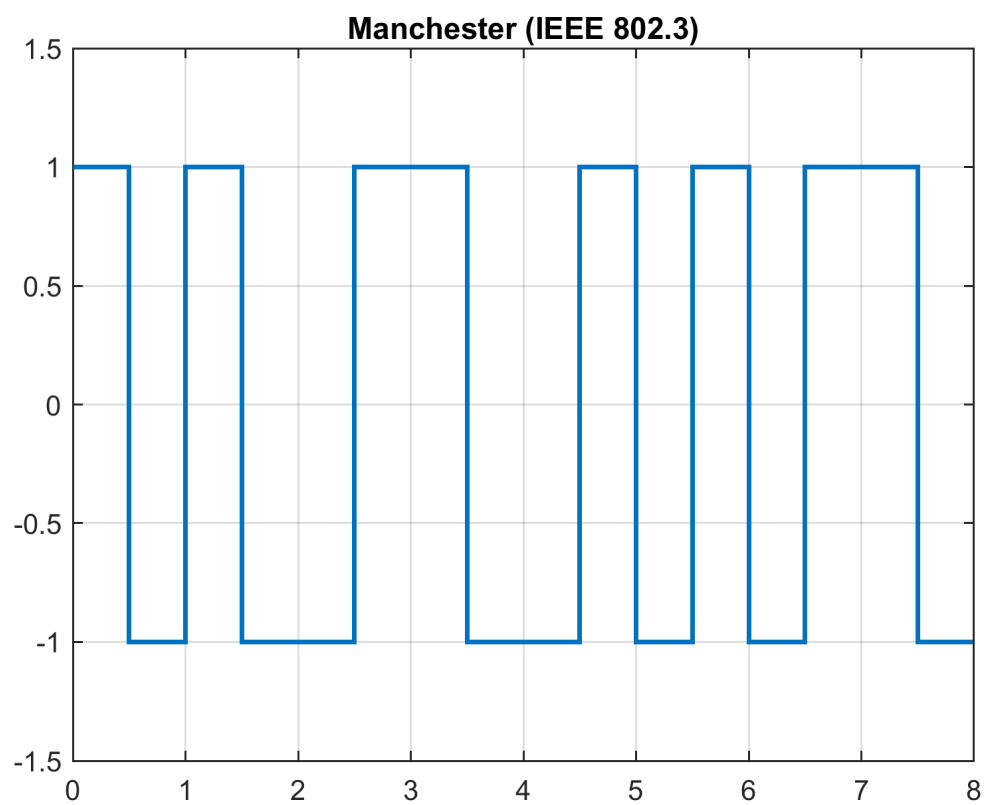


Рис. 8. Манчестерский код (по стандарту IEEE 802.3)

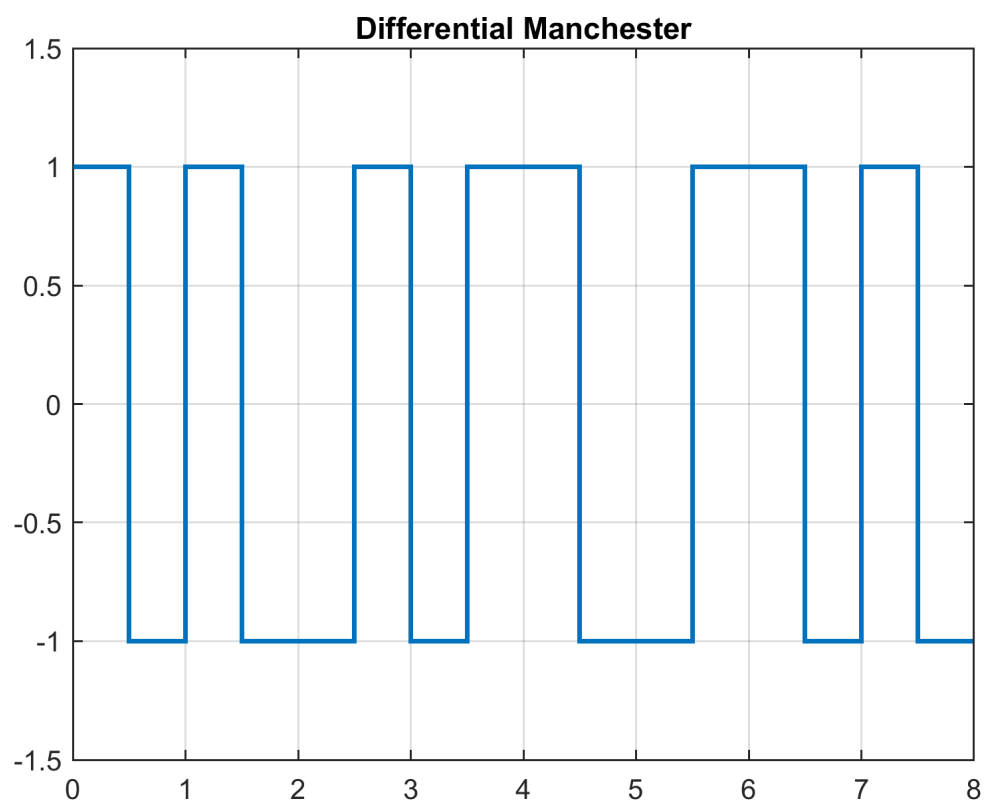


Рис. 9. Манчестерский дифференциальный код